



Efficient Algorithms to Compute Closed Substrings

Samkith K Jain¹ · Neerja Mhaskar¹

Received: 23 December 2025 / Accepted: 25 August 2026

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

Abstract

A closed string u is either of length one or contains a non-empty border that occurs only as a prefix and as a suffix in u and nowhere else within u . In this paper, we present fast $\mathcal{O}(n \log n)$ time algorithms to compute all $\mathcal{O}(n^2)$ closed substrings by introducing a compact representation for all closed substrings of a string $w[1..n]$, using only $\mathcal{O}(n \log n)$ space. These simple and space-efficient algorithms also compute maximal closed strings. Furthermore, we compare the performance of these algorithms and identify classes of strings where each performs best. Finally, we show that the exact number of maximal closed substrings in a Fibonacci word f_n , for $n \geq 5$, is $\approx \left(1 + \frac{1}{\phi^2}\right) F_n \approx 1.382 F_n$, where ϕ is the golden ratio and $F_n = |f_n|$.

Keywords Closed strings · Maximal closed substrings · Fibonacci words

1 Introduction

A closed string u is either of length one or contains a non-empty border that occurs only as a prefix and as a suffix in u and nowhere else within u ; otherwise, it is called open. In other words, the longest (possibly overlapping) border of a closed substring u does not have internal occurrences in u . For example, in the string $abacab$, ab occurs only as a prefix and a suffix. A maximal closed substring (MCS) is a closed substring that cannot be extended by one character to either the left or the right so as to form another closed substring. For example, in string $w[1..8] = abaccaba$, the MCSs are $w[1..8] = abaccaba$, $w[3..6] = acca$, $w[1..3] = w[6..8] = aba$, $w[4..5] = cc$ (five in total) and all occurrences of a and b , while the substrings $w[2..7] = baccab$, $w[2..8] = baccaba$, $w[1..7] = abaccab$, and both occurrences of c are closed but not maximal.

✉ Neerja Mhaskar
pophlin@mcmaster.ca
Samkith K Jain
kishors@mcmaster.ca

¹ Department of Computing and Software, McMaster University, 1280 Main St W, Hamilton L8S 4L8, Ontario, Canada

The concept of open and closed strings was introduced by Fici in [1] to study the combinatorial properties of strings, with connections to Sturmian and Trapezoidal words. Then concepts of *Longest Closed Factorization* and *Longest Closed Factor Array* were introduced in [2], with improved solutions later proposed in [3]. Later, [4] introduced efficient algorithms for minimal and shortest closed factors. Related problems such as the k -closed string problem [5] and combinatorial problems on closed factors in Arnoux-Rauzy and m -bonacci words [6, 7] have also been explored.

It has been shown that a string contains at most $\mathcal{O}(n^2)$ distinct closed factors [8]. More recently (in 2024), [9] showed that the maximal number of distinct closed factors in a finite word of length n is at most $\frac{n^2}{6}$. In early 2025, [10] presented a linear algorithm to count the number of distinct closed factors of a string in $\mathcal{O}(n \log \sigma)$ time and another in $\mathcal{O}(n)$ time. They also described methods to enumerate all closed and distinct closed factors, achieving a complexity of $\mathcal{O}(n^2)$, which is further optimized to $\mathcal{O}(n\sqrt{\log n} + \text{output} \cdot \log n)$ for distinct closed substrings, using weighted ancestor queries, where output is the number of distinct closed substrings.

In [11] Badkobeh et al. introduced the notion of MCSs and proposed a $\mathcal{O}(n\sqrt{n})$ loose upper bound on the number of MCSs in a string of length n . They also proposed an algorithm using suffix trees to compute the MCSs for strings on a binary alphabet in $\mathcal{O}(n \log n + m)$ time, where m is the number of MCSs. Badkobeh et al. in [12] extended their algorithm to a general alphabet with a time complexity of $\mathcal{O}(n \log n)$, implicitly improving the bound on the total number of MCSs, this was later directly proved by Kosolobov in [13].

In this paper, we propose solutions to the following problems:

Compact Representation of All $\mathcal{O}(n^2)$ Closed Substrings: Given a string $w[1..n]$ we define a compact representation of all $\mathcal{O}(n^2)$ closed substrings of w using $\mathcal{O}(n \log n)$ space.

Compute All $\mathcal{O}(n^2)$ Closed Substrings: Given a string $w[1..n]$, we compute all $\mathcal{O}(n^2)$ closed substrings of w in a compact representation in $\mathcal{O}(n \log n)$ time and $\mathcal{O}(n)$ auxiliary space.

Compute All MCSs: Given a string $w[1..n]$ we find all MCSs of w in $\mathcal{O}(n \log n)$ time and $\mathcal{O}(n)$ auxiliary space.

Exact Number of MCSs in a Fibonacci Word: Given a Fibonacci word f_n , we give an exact equation to find the number of MCSs in f_n which is denoted by $M(f_n)$ and is defined in (8). We also show that $M(f_n) \approx 1.382F_n$.

In Section 2, we introduce the terminology and data structures used in the paper. In Section 3 we define a compact representation of all closed substrings of a given string $w[1..n]$. In Section 4, we present two algorithms to compute all closed substrings, including all MCSs. In Section 5 we provide a formula to compute the exact number of MCSs in a Fibonacci word. We then present experimental analysis of the proposed algorithms in Section 6. Finally, we conclude with future work and open problems in Section 7.

An earlier and abridged form of this work appeared in *SPIRE 2025* [14]. In this extended version, we present an alternate algorithm to compute all closed substrings, including all MCSs, analyze the space complexities of both algorithms, and provide extensive experimental results comparing their performance across different classes of strings.

2 Preliminaries

An alphabet Σ , of size σ , is a set of symbols that is totally ordered. A string (**word**), written as $w[1..n]$, is an element of Σ^* , whose length is represented by n . The **empty string** of length zero is denoted by ε . A **substring (factor)** $w[i..j]$ of w is a string, where $1 \leq i, j \leq n$, if $i > j$ then the substring is ε . A substring $w[i..j]$ where $i = 1$ ($j = n$) is called a **prefix (suffix)** of w ; it is a **proper prefix (suffix)** of w , if $j - i + 1 < n$. If a substring u of w is both a proper prefix and a proper suffix of w , then it is called a **border**. A **cover** of a string w is a substring u of w such that w can be constructed by overlapping and/or adjacent instances of u . If $|u| < |w|$, then it is said to be a **proper cover** (for more details on covers, see the survey [15]).

We now formally define the notions of closed strings and maximal closed strings introduced earlier. Let $k = \max\{i \mid 0 \leq i < n, w[1..i] = w[n - i + 1..n]\}$ be the maximum border length of w . Then w is **closed** if and only if $n = 1$ or ($k \geq 1$ and $\forall j \in [2..n - k]$, $w[1..k] \neq w[j..j + k - 1]$). Otherwise, w is **open**. An occurrence $w[i..j]$ of a substring u in $w[1..n]$ is **maximal right-closed** if u is closed and it cannot be extended to the right into another closed substring; that is, either $j = n$ or $w[i..j + 1]$ is open. Symmetrically, the occurrence is **maximal left-closed** if u is closed and either $i = 1$ or $w[i - 1..j]$ is open. An occurrence $w[i..j]$ is a **maximal closed substring (MCS)** of w if it is both maximal left-closed and maximal right-closed; that is, u is closed, and ($i = 1$ or $w[i - 1..j]$ is open) and ($j = n$ or $w[i..j + 1]$ is open). Hence, every MCS is a maximal right-closed substring, but the converse does not hold, since a maximal right-closed substring need not be maximal left-closed. An MCS of length one is called a **singleton** MCS; otherwise, it is called a **non-singleton** MCS. For example, in the string $w[1..8] = abaccaba$, the non-singleton MCSs are $w[1..8] = abaccaba$, $w[3..6] = acca$, $w[1..3] = w[6..8] = aba$, and $w[4..5] = cc$, while the singleton MCSs are all occurrences of a and b .

An integer p with $1 \leq p \leq n$ is called a **period** of $w[1..n]$ if $w[i] = w[i + p]$, for all $1 \leq i \leq n - p$. We denote the **smallest period** of w as $per(w)$. A string w is **periodic** if $per(w) \leq |w|/2$. A substring $w[i..j]$ is a **repetition** in w , if it is of the form u^e for some non-empty string u , and integer $e \geq 2$; otherwise it is **primitive**. A substring $w[i..j]$ in a string w is a **maximal repetition** if $w[i..j]$ is a repetition of the form u^e , u is primitive ($|u| = t = per(w[i..j])$), and ($i \leq t$ or $w[i - t..i + t(e - 1) - 1] \neq u^e$) and ($j + t > n$ or $w[i + t..i + (e + 1)t - 1] \neq u^e$). A **run** is a periodic substring $w[i..j] = u^e u'$ of $w[1..n]$, where u is primitive ($|u| = t = per(w[i..j])$), u' is a (possibly empty) proper prefix of u and $e \geq 2$, whose periodicity cannot be extended to the left or to the right; that is, $i = 1$ or $w[i - 1] \neq w[i + t - 1]$, and $j = n$ or $w[j + 1] \neq w[j + 1 - t]$.

A **gapped repeat** is a substring of $w[1..n]$ of the form uvu , where the two occurrences of u are non-empty and are called the **arms**, v is called the **gap**, and the number $|u| + |v|$ is the period. An occurrence of a gapped repeat is called a **maximal gapped repeat** if it cannot be extended to the left or to the right while preserving its period. A maximal gapped repeat that is closed is referred to as a **gapped-MCS**.

In the literature, a gapped repeat is formally a pair consisting of a substring together with its period $|u| + |v|$, since the same substring may admit several decompositions uvu with different periods. Instead, we refer to a gapped repeat simply as a substring

of the form uvu . This convention differs slightly from the standard one, but suffices for our purposes.

A closed substring $u = w[i..j]$ of a string $w[1..n]$ is said to be **right-extendible** to a maximal right-closed substring $r = w[i..j']$ if $j < j' \leq n$ and every prefix $w[i..k]$ of r with $j < k < j'$ is also closed. In this case, r is the unique maximal right-closed extension of u .

The **suffix array** $SA[1..n]$ of $w[1..n]$ is an integer array of length n , where each entry $SA[i]$ points to the starting position of the i -th lexicographically least suffix of w . The **longest common prefix array** $LCP[1..n]$ of $w[1..n]$ is an array of integers such that $LCP[i]$ is equal to the length of the longest common prefix of suffixes starting at positions $SA[i - 1]$ and $SA[i]$ for all $2 \leq i \leq n$, we assume $LCP[1] = 0$.

2.1 Overview of Crochemore's Maximal Repetition Algorithm

We provide a brief overview of Crochemore's well-known algorithm for computing maximal repetitions of a given string w [16]—which we refer to as **CMR Algorithm**, while highlighting key facts that form the basis of the algorithm. This algorithm serves as the basis for our Algorithm 3.

Fact 1 (Levels): The **CMR** algorithm operates on levels $k = 1, 2, \dots, n$, where $|w| = n$. At each level k , it maintains a collection of disjoint sets C_k of index positions corresponding to starting positions of substrings of w of length k , grouped according to substring equality.

Let $1 \leq k \leq n$ denote a level in the CMR Algorithm. An equivalence relation $\approx_k$ on the set of positions $\{1, 2, \dots, n\}$ is defined by

$$i \approx_k j \iff w[i..i+k-1] = w[j..j+k-1],$$

where $i, j \in \{1, 2, \dots, n\}$ are distinct index positions. The corresponding equivalence classes of $\approx_k$ form a collection of disjoint sets whose union is $\subseteq \{1, 2, \dots, n\}$. For each position i , we denote its equivalence class by

$$[i]_k = \{j \mid j \approx_k i\}$$

Thus, each class $[i]_k$ consists of the starting positions of all occurrences of a substring of length k in w . Let U_k be the set of all distinct substrings of w of length k . Then, the set of equivalence classes at level k is defined as

$$C_k = \left\{ \{j \mid w[j..j+k-1] = u\} \mid u \in U_k \right\}.$$

Figure 2 shows the classes of $\approx_k$ where $1 \leq k \leq 5$ for the string $w[1..11] = \text{mississippi}$.

Fact 2 (Class Representation). For each $k = 1, 2, \dots, n$, all sets in C_k are stored as ordered lists. Consequently, in any class $C' \in C_k$, the index positions appear in sorted order. The ordered list representation is implemented using doubly linked lists with auxiliary pointers to elements, along with additional auxiliary data structures, allowing constant-time insertion, deletion, and next and previous element lookup.

Fact 3 (Refinement): The equivalence classes at level $k + 1$, for $1 \leq k \leq n - 1$, are computed from those at level k through a process called *refinement*.

Initially, the equivalence classes at level $k = 1$ are obtained by scanning the string w from left to right and grouping together positions that contain the same symbol in Σ . Therefore, at $k = 1$, we obtain σ classes. To compute the classes at level $k + 1$, each class in C_k is *refined* as follows: let $C', C'' \in C_k$ such that $C' \neq C''$. For a class C' , consider indices $i, j \in C'$ such that $i \neq j$. If $i + 1, j + 1 \in C''$, then i and j remain in the same class; otherwise they are separated or *refined out* into different possibly new classes.

Once we refine all classes at level k , we get all the classes at level $k + 1$. The refinement process to compute equivalence classes at various levels for the string $w = \text{mississippi}$ is illustrated with arrows in Fig. 2.

Fact 4 (Termination): Once a class becomes a singleton (of size one), it cannot be refined further. The algorithm terminates when all classes are singletons.

The $\mathcal{O}(n \log n)$ time complexity, in Lemma 1, lies in the choice of classes for refinement. For each family of classes (classes that were formed as a refinement of a class on the previous level), we identify the largest class and call all the others *small*. By using only small classes for refinement to get all the equivalence classes in the next level, $\mathcal{O}(n \log n)$ complexity is achieved, as each element belongs only to $\mathcal{O}(\log n)$ small classes. To support efficient refinement operations, the classes are maintained as ordered doubly linked lists, allowing elements to be removed from one class and inserted into another in constant time. For a detailed description of the CMR algorithm and its complexity, we refer the reader to [16].

Lemma 1 (Theorem 7 in [16]) *The CMR Algorithm correctly computes all the maximal repetitions in a string $w[1..n]$ in $\mathcal{O}(n \log n)$ time.*

3 Compact Representation of all Closed Substrings

A string $w[1..n]$ contains $\mathcal{O}(n^2)$ closed substrings, which can naively be computed in $\mathcal{O}(n^2)$ time by computing the border of all substrings of w . In this section, we define a compact representation for all $\mathcal{O}(n^2)$ closed substrings in Theorem 1 that requires only $\mathcal{O}(n \log n)$ space. Algorithm 4 in Section 4 presents an $\mathcal{O}(n \log n)$ time solution to compute all closed substrings of w in a compact representation of size $\mathcal{O}(n \log n)$.

Let u and v be proper closed prefixes of the maximal right-closed substring r . Then, by definition of right-extendibility we define an equivalence relation R as follows:

$$R = \{(u, v) \mid u \text{ and } v \text{ are both right-extendible to } r\}.$$

Next, we define the equivalence class E_r of r under the relation R as follows:

$$E_r = \{u \mid u \text{ is right-extendible to } r\}. \quad (1)$$

Note that by definition of right-extendibility r does not belong to E_r .

Lemma 2 Given a string $w[1..n]$, if $w[1..m]$ is the shortest maximal right-closed prefix of w , then $w[1..m] = \lambda^m$, where $\lambda \in \Sigma$ and $1 \leq m \leq n$. Moreover, each prefix of λ^m is also closed.

Proof We prove the lemma in two parts. First part by contradiction. Suppose that $w[1..m]$ is the shortest maximal right-closed prefix of w , and that $w[1..m] \neq \lambda^m$. W.l.o.g let $w[1] = \lambda$. Let $1 < j \leq m$ be the smallest index such that $w[j] = \lambda_1$ and $w[1..j-1] = \lambda^{j-1}$, where $\lambda_1 \neq \lambda \in \Sigma$. Then, $w[1..j] = \lambda^j \lambda_1$. However, by definition of maximal right-closed string, $w[1..j-1] = \lambda^{j-1}$ is a shorter maximal right-closed string of w — a contradiction.

Second, for any string of the form $u = \lambda^m$, consider its prefixes $p = u[1..j]$, where $1 \leq j \leq m$. If $j = 1$, then p is closed trivially. If $j \geq 2$, then the border length of each prefix p is equal to $j - 1$, which is maximum and unique. Thus, all the prefixes of $u = \lambda^m$ are closed. $\square$

Let r denote a maximal right-closed substring, b its border, and p its shortest closed prefix such that $p = r$ or $p \in E_r$. Let $r_1, r_2, \dots, r_{K_i}$ be all the maximal right-closed substrings starting at index i of a string $w[1..n]$, where $1 \leq i \leq n$, and K_i is the number of maximal right-closed substrings starting at index i , such that $|r_1| < |r_2| < \dots < |r_{K_i}|$.

In (2) we present a formula to compute the length of the shortest closed prefix p_j of r_j such that $p_j = r_j$ or $p_j \in E_{r_j}$, where $1 \leq j \leq K_i$. Lemma 3 proves (2) and is illustrated in Fig. 1.

Lemma 3 Let $1 \leq j \leq K_i$. The length of the shortest closed prefix p_j of r_j , such that $p_j = r_j$ or $p_j \in E_{r_j}$, is given by:

$$|p_j| = \begin{cases} 1, & \text{if } j = 1, \\ |r_j| - |b_j| + |b_{j-1}| + 1, & \text{if } 1 < j \leq K_i, \end{cases} \quad (2)$$

where b_j and b_{j-1} are the longest borders of r_j and r_{j-1} , respectively.

Proof We prove the lemma by considering two cases. For $j = 1$, by (1) and Lemma 2, all prefixes of r_1 are closed and $|p_1| = 1$.

For $1 < j \leq K_i$, by definition of a closed substring, the shortest closed prefix p_j starting at index i and longer than r_{j-1} (whose longest border has length $|b_{j-1}|$) must

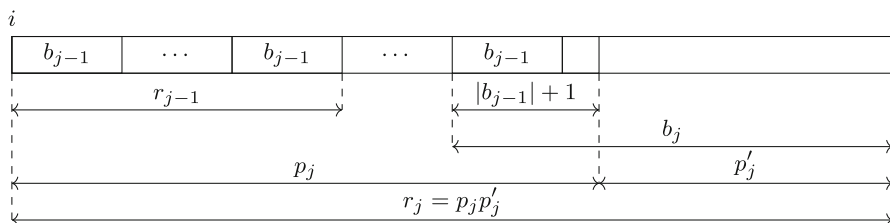


Fig. 1 Illustration of the second case in the proof of Lemma 3, showing that $|p_j|$ can be computed from $|b_j|$, $|b_{j-1}|$ and $|r_j|$, such that $p_j = r_j$ or $p_j \in E_{r_j}$. Note that when $p_j = r_j$, $p'_j = \varepsilon$

have the longest border of length exactly $|b_{j-1}| + 1$. Either $p_j = r_j$ or by definition of right-extendibility p_j extends to $r_j = p_j p'_j$, with the longest border b_j , and each prefix of r_j with length at least $|p_j|$ is closed. Since $|p'_j| = |b_j| - (|b_{j-1}| + 1)$, we have $|r_j| = |p_j| + |p'_j| = |p_j| + |b_j| - (|b_{j-1}| + 1)$. Solving for $|p_j|$, we obtain the desired formula: $|p_j| = |r_j| - |b_j| + |b_{j-1}| + 1$. $\square$

Lemma 4 (Theorem 1 in [13]) *A string $w[1..n]$ contains at most $\mathcal{O}(n \log n)$ maximal right (left)-closed substrings.*

Theorem 1 follows directly from (1), Lemma 3 and 4.

Theorem 1 *The compact representation of all closed substrings of $w[1..n]$ requires at most $\mathcal{O}(n \log n)$ space and is given by:*

$$\mathcal{C}(w) = \{(i, |p_j|, |r_j|) \mid 1 \leq i \leq n, 1 \leq j \leq K_i\}, \quad (3)$$

where K_i is the number of maximal right-closed substrings starting at position i , and for each j -th such substring r_j , starting at index i , p_j is the shortest closed prefix of r_j such that $p_j = r_j$ or $p_j \in E_{r_j}$.

The set of all closed substrings of $w[1..n]$, denoted by $\text{Closed}(w)$, can be enumerated using the following equation:

$$\text{Closed}(w) = \bigcup_{(i, |p_j|, |r_j|) \in \mathcal{C}(w)} \{w[i \dots i + \ell - 1] \mid \ell \in [|p_j|, |r_j|]\}. \quad (4)$$

An example of $\mathcal{C}(w)$ and the enumeration $\text{Closed}(w)$ for the string $w[1..11] = \text{mississippi}$ is given in Table 2.

4 Algorithms for Closed Substrings and MCSs

In this section, we introduce the primary data structure—the *MRC* array—which is used to compute all closed substrings in a compact representation and MCSs in $\mathcal{O}(n \log n)$ time. See Table 1 for an example of the *MRC* array for the string $w[1..11] = \text{mississippi}$. We present two algorithms: Algorithm 2 based on SA & LCP array and Algorithm 3 based on Crochmore's equivalence classes approach.

Definition 1 (*MRC Array*¹) Given a string $w[1..n]$, the **maximal right-closed array** (*MRC*) is an array of size n , where $\text{MRC}[i]$, $1 \leq i \leq n$, stores the list of ordered pairs $(|r_j|, |b_j|)$, where $j = K_i$ to 1.

¹ This definition uses the notation introduced in Section 3.

Table 1 The $\mathcal{MRC}$ array for the string $w[1..11] = \text{mississippi}$.

Index (i)	$\mathcal{MRC}[i]$
1	(1, 0)
2	(7, 4), (1, 0)
3	(6, 3), (2, 1)
4	(5, 2), (3, 1), (1, 0)
5	(4, 1), (1, 0)
6	(2, 1)
7	(1, 0)
8	(4, 1), (1, 0)
9	(2, 1)
10	(1, 0)
11	(1, 0)

4.1 Computing $\mathcal{MRC}$ Array using SA and LCP

Algorithm 2 computes the $\mathcal{MRC}$ array by first identifying all the maximal right-closed substrings of length greater than one. It uses a stack to store AVL trees created during its execution. The algorithm scans the LCP array from left to right. Beginning with $\text{LCP}[1] = 0$, and for increasing $\text{LCP}[i]$ values, the algorithm creates a single node AVL tree with $\text{key} = \text{SA}[i]$ and $\text{value} = \text{LCP}[i]$ and pushes it onto the stack, until either the LCP value decreases or the end of the array is reached.

We denote by $\text{LCP}_{\max}$ the value stored in the root node of the AVL tree on the top of the stack. When a decrease in the LCP value (or the end of the array) is encountered, the algorithm pops a collection of AVL trees from the stack, and store them in $\mathcal{L}_{\text{suffixSets}}$. Note that all trees in $\mathcal{L}_{\text{suffixSets}}$ have the following property: the suffixes represented by the keys in each AVL tree share a common prefix of length $\text{LCP}_{\max}$. Conversely, any suffix of w that shares a common prefix of length $\text{LCP}_{\max}$ with the suffixes represented in $\mathcal{L}_{\text{suffixSets}}$ is represented by a node in one of the AVL trees in $\mathcal{L}_{\text{suffixSets}}$. Moreover, since $\text{LCP}_{\max}$ is the maximum LCP value among all the AVL trees on the stack, this collection stored in the list $\mathcal{L}_{\text{suffixSets}}$, preserves the strictly decreasing order of ordered pairs in the $\mathcal{MRC}$ array. This collection is then merged into a single AVL tree using the UNION operation presented in [17]², resulting in $\mathcal{T}_{\text{suffixes}} = \bigcup \mathcal{L}_{\text{suffixSets}}$.

We minimally modify the UNION operation to also output a list of ordered pairs representing maximal right-closed substrings via the CHANGLIST operation defined in 5. Let $\mathcal{P}' \subseteq [1..n]$, and let $\mathcal{P} = \{P_1, P_2, \dots, P_t\}$ be a partition of $\mathcal{P}'$; that is, $\bigcup_{i=1}^t P_i = \mathcal{P}'$, $P_i \cap P_j = \emptyset$ for $i \neq j$, and $P_i \neq \emptyset$ for all i . For $X \subseteq \mathbb{Z}^+$ and $x < \max(X)$, define $\text{next}_X(x) = \min\{y \in X \mid y > x\}$. For $x = \max(X)$, $\text{next}_X(x)$ is not defined. We then define:

$$\text{ChangeList}(\mathcal{P}, \mathcal{P}') = \{(x, \text{next}_{\mathcal{P}'}(x)) \mid \text{next}_{P_i}(x) \neq \text{next}_{\mathcal{P}'}(x), x \in P_i\}, \quad (5)$$

² The UNION operation, defined in [17], builds upon foundational methods from [18, 19].

where i is the unique index with $x \in P_i$.

By construction, the AVL trees stored in $\mathcal{L}_{\text{suffixSets}}$ form a partition of the set of suffix positions represented by $\mathcal{T}_{\text{suffixes}}$. Hence, $\mathcal{L}_{\text{suffixSets}}$ and $\mathcal{T}_{\text{suffixes}}$ correspond to the partition $\mathcal{P}$ and the set $\mathcal{P}'$, respectively, in the definition of CHANGELIST. Each pair $(x, y) \in \text{ChangeList}(\mathcal{L}_{\text{suffixSets}}, \mathcal{T}_{\text{suffixes}})$ identifies a maximal right-closed substring $r_j = w[x..y + \text{LCP}_{\max} - 1]$, where $\text{LCP}_{\max}$ is the length of its longest border. Hence, the pair $(y + \text{LCP}_{\max} - x, \text{LCP}_{\max})$ is inserted into $\text{MRC}[x]$. Since $\text{MRC}[x]$ is a list, the insert operation is performed in constant time. All such pairs from the CHANGELIST are inserted into the MRC array. The algorithm then assigns to the root node of the AVL tree, $\mathcal{T}_{\text{suffixes}}$, the LCP value of the AVL tree at the top of the stack and pushes it onto the stack, only if the stack is not empty—thus maintaining the lexicographic ordering of the set of suffixes—and continues scanning the LCP array.

To compute the maximal right-closed substrings of length one, it performs a simple linear scan of the string w and adds $w[i]$ to $\text{MRC}[i]$, if $i = n \vee (1 \leq i < n \wedge w[i] \neq w[i + 1])$.

It is known that LCP array identifies all the occurrences of the repeating substrings that cannot be extended to the right. Moreover, the suffix array groups all these occurrences within specific ranges. Algorithm 2 identifies such ranges by tracking the rise and fall of values in the LCP array, and stores the corresponding indices in $\mathcal{L}_{\text{suffixSets}}$ as keys in AVL trees, starting with the substring of length $\text{LCP}_{\max}$. The UNION operation then merges these single node AVL trees to form one AVL tree ($\mathcal{T}_{\text{suffixes}}$) that contains all the SA values; that is, all the starting positions of the substrings of length $\text{LCP}_{\max}$ in the string w .

The $\text{CHANGELIST}(\mathcal{L}_{\text{suffixSets}}, \mathcal{T}_{\text{suffixes}})$ operation then returns every pair of consecutive occurrences of substrings in the $\mathcal{L}_{\text{suffixSets}}$. These consecutive occurrences form the border of the maximal right-closed substring and are therefore added to the MRC array. Finally, a simple linear scan adds maximal right-closed substrings of length one, as shown in the ADDSINGLETONMRC procedure presented in Algorithm 1. The algorithm clearly computes the maximal right-closed substrings in strictly decreasing order of length as required in the MRC array. Thus, the correctness of Algorithm 2 follows.

Algorithm 1 Add all maximal right-closed substrings of length 1 to the MRC array.

```

1: procedure  $\text{ADDSINGLETONMRC}(w[1..n], \text{MRC})$ 
2:   for  $i \leftarrow 1$  to  $n$  do
3:     if  $i = n$  or  $w[i] \neq w[i + 1]$  then
4:        $\text{insert}(\text{MRC}[i], (1, 0))$ 

```

In Algorithm 2, the maximum size of the stack is at most n . Moreover, the total space occupied by all the AVL trees is also n , as the AVL trees form a disjoint partition of suffixes starting at index positions 1 to n . Therefore, the maximum size of the structure $\mathcal{L}_{\text{suffixSets}}$ is bounded by n . The procedure $\text{CHANGELIST}(\mathcal{L}_{\text{suffixSets}}, \mathcal{T}_{\text{suffixes}})$ can return a list of at most n pairs because no substring of $w[1..n]$ can repeat more than n times. Hence, Algorithm 2 requires $\mathcal{O}(n)$ auxiliary space.

We note that the algorithm in the worst case performs $4n - 2$ stack operations for a string $w = \lambda^n$ and in the best case performs $2n$ stack operations for a string with all distinct characters.

Algorithm 2 Compute $\mathcal{MRC}$ Array of $w[1..n]$ using its SA and LCP

```

1: procedure COMPUTE- $\mathcal{MRCARRAY1}(w, SA, LCP)$ 
2:    $i \leftarrow 0$ ;  $\mathcal{MRC} \leftarrow \{\emptyset\}^n$ ;  $stack \leftarrow \emptyset$ 
3:   while  $stack \neq \emptyset$  or  $i < n$  do
4:     while  $i < n$  and  $(stack = \emptyset \text{ or } LCP[i] \geq LCP(top(stack)))$  do
5:        $push(stack, AVLTree(\{SA[i]\}, LCP[i]))$ 
6:        $i \leftarrow i + 1$ 
7:      $\mathcal{L}_{suffixSets} \leftarrow \emptyset$ 
8:      $LCP_{max} \leftarrow LCP(top(stack))$ 
9:     while  $stack \neq \emptyset$  and  $LCP(top(stack)) = LCP_{max}$  do
10:       $insert(\mathcal{L}_{suffixSets}, pop(stack))$ 
11:     if  $LCP_{max} \neq 0$  then
12:        $previousLCP \leftarrow LCP(top(stack))$ 
13:        $insert(\mathcal{L}_{suffixSets}, pop(stack))$ 
14:        $\mathcal{T}_{suffixes} = \bigcup \mathcal{L}_{suffixSets}$   $\triangleright \bigcup \mathcal{L}_{suffixSets}$  returns an AVL tree
15:       foreach  $(x, y) \in \text{CHANGEList}(\mathcal{L}_{suffixSets}, \mathcal{T}_{suffixes})$  do  $\triangleright$  Equation (5)
16:          $insert(\mathcal{MRC}[x], (y + LCP_{max} - x, LCP_{max}))$ 
17:          $LCP(\mathcal{T}_{suffixes}) \leftarrow previousLCP$ 
18:          $push(stack, \mathcal{T}_{suffixes})$ 
19:    $\text{ADDSINGLETONMRC}(w, \mathcal{MRC})$   $\triangleright$  See Algorithm 1
20:   return  $\mathcal{MRC}$ 

```

Lemma 5 ([17]) *The UNION operation correctly computes the ChangeList, and any sequence of UNION operations takes $\mathcal{O}(n \log n)$ time in total.*

Theorem 2 *Algorithm 2 correctly computes the $\mathcal{MRC}$ array of a string $w[1..n]$ using SA and LCP in $\mathcal{O}(n \log n)$ time and $\mathcal{O}(n)$ auxiliary space.*

Proof We prove the total time complexity of Algorithm 2 in three parts. First, for stack operations, we introduce a potential function $\Phi(S) = |S| \geq 0$, where $|S|$ is the number of elements in the stack. The actual cost of each push and pop operation is 1. Since we push all n single node AVL trees corresponding to each suffix onto the stack, the amortized cost for each push operation is $c'_{push} = 1 + \Delta\Phi(S) = 2$, resulting in a total amortized cost of $2n$. Additionally, in the algorithm, we pop at least two AVL trees and push their UNION back onto the stack, and because the amortized cost of this sequence of pops and push in the worse case is $c'_{pops-push} = 3 + \Delta\Phi(S) = 2$. The maximum number of such operations is $n - 1$, leading to a total amortized cost of $2(n - 1)$. Therefore, the total cost of all stack operations in the worst case is $4n - 2$, which is $\mathcal{O}(n)$. Second, by Lemma 5, the time complexity of the sequence of UNION operations is $\mathcal{O}(n \log n)$. Finally, the single-characters that are maximal right-closed are computed by a linear scan on w , taking $\mathcal{O}(n)$ time. Therefore, the total time complexity is $\mathcal{O}(n \log n)$. $\square$

4.2 Computing $\mathcal{MRC}$ Array using Equivalence Classes [16]

Algorithm 2 requires the pre-processed SA and LCP arrays to compute the $\mathcal{MRC}$ array of a string $w[1..n]$. In this section, we present an alternative algorithm that computes the $\mathcal{MRC}$ array directly from the input string $w[1..n]$, without any pre-processing, by modifying the CMR Algorithm that uses the notion of equivalence classes.

The CMR Algorithm identifies the starting index positions of all unique substrings of $w[1..n]$ of length k , for increasing values of k starting with $k = 1$. The combined occurrences of substrings at consecutive index positions within a class $C' \in C_k$ then form a closed substring (not necessarily a maximal right-closed substring). We first describe how to identify all maximal right-closed substrings from the equivalence classes across all the k levels in the CMR Algorithm, and then present constant-time modifications to it that enable computation of the $\mathcal{MRC}$ array in $\mathcal{O}(n \log n)$ time.

Recall from Section 4.1 that, $next_X(x) = \min\{y \in X \mid y > x\}$, where $X \subseteq \mathbb{Z}^+$ and $x < \max(X)$. Similarly, we define $prev_X(x) = \max\{y \in X \mid y < x\}$, where $x > \min(X)$. For $x = \min(X)$, $prev_X(x)$ is not defined. Consider a class $C' \in C_k$. For an index $i \in C'$, the value $next_{C'}(i)$ (if defined) denotes the successor of i , that is, the smallest element greater than i in the sorted class list C' . Let $j = next_{C'}(i)$. Equivalently, $i = prev_{C'}(j)$. The pair (i, j) forms a closed substring starting at position i with a border of length k . Note that both $next_{C'}()$ and $prev_{C'}()$ can be computed in constant time for any class $C' \in C_k$, as shown in [16].

At the next level $k + 1$, suppose $i \in C''$ for some $C'' \in C_{k+1}$ but $j \notin C''$. This implies that the new characters appended to substrings of length k at positions i and j are not the same, and therefore the border of length k cannot be extended to the right. Hence, the substring starting at index position i of length $j + k - i$ is a maximal right-closed substring. Conversely, suppose that $w[i..j + k - 1]$ is a maximal right-closed substring whose longest border has length k . Then i and j belong to the same equivalence class $C' \in C_k$, and $j = next_{C'}(i)$. For example, in Fig. 2, which illustrates

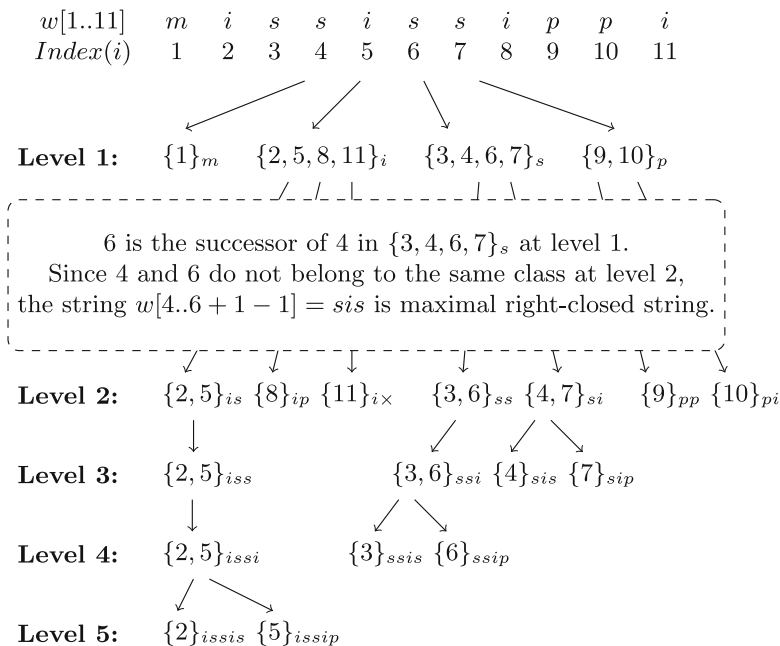


Fig. 2 Equivalence classes across successive levels in the CMR Algorithm for the string $w[1..11] = mississippi$, illustrating the identification of maximal right-closed substrings from level transitions

the equivalence classes for the string $w[1..11] = \text{mississippi}$, 6 is the successor of 4 in the class $\{3, 4, 6, 7\}_s$ at level 1. Since 4 and 6 do not belong to the same class at level 2, the substring $w[4..6 + 1 - 1] = \text{sis}$ is a maximal right-closed substring with unique border $w[4..4] = [6..6] = s$ of length $k = 1$.

Algorithm 3 Compute $\mathcal{MRC}$ array of $w[1..n]$ without pre-processing w .

```

1: procedure COMPUTE- MRCARRAY2( $w$ )
2:    $\mathcal{MRC} \leftarrow \{\emptyset\}^n$ 
3:   ADDSINGLETONMRC( $w, \mathcal{MRC}$ ) ▷ See Algorithm 1
4:    $T[1..n] \leftarrow \{\emptyset\}^n; k \leftarrow 1$ 
5:    $C_1 \leftarrow$  Equivalence classes at level 1, all marked small.
6:   while level  $k$  contains small classes do
7:     During the refinement of a class  $C' \leftarrow \{j_1, j_2, j_3, \dots, j_{|C'|}\} \in C_k$ 
8:       while  $j_m \in C' \mid m \in \{1, 2, 3 \dots |C'|\}$  is refined out of  $C'$  do
9:         if  $prev_{C'}(j_m)$  exists then
10:            $j_{m-1} \leftarrow prev_{C'}(j_m)$ 
11:           if  $T[j_{m-1}] \neq k$  and  $w[j_{m-1}..j_m + k - 1]$  is maximal right-closed then
12:              $insert(\mathcal{MRC}[j_{m-1}], (j_m + k - j_{m-1}, k))$ 
13:            $T[j_{m-1}] \leftarrow k$ 
14:         if  $next_{C'}(j_m)$  exists then
15:            $j_{m+1} \leftarrow next_{C'}(j_m)$ 
16:           if  $T[j_m] \neq k$  and  $w[j_m..j_{m+1} + k - 1]$  is maximal right-closed then
17:              $insert(\mathcal{MRC}[j_m], (j_{m+1} + k - j_m, k))$ 
18:            $T[j_m] \leftarrow k$ 
19:          $k \leftarrow k + 1$ 
20:       Compute small classes at level  $k$ 
21:   return  $\mathcal{MRC}$ 

```

Given a string $w[1..n]$, we first compute the maximal right-closed substrings of length 1 and insert them into the $\mathcal{MRC}$ array, as shown in the ADDSINGLETON-MRC procedure presented in Algorithm 1. To compute all the maximal right-closed substrings of length greater than 1 we adapt the CMR Algorithm to use an auxiliary *boolean* array $T_k[1..n]$ at each level k , where $T_k[i]$ records whether i has a successor in its class $C' \in C_k$. More precisely, for all $1 \leq i \leq n$, $T_k[i] = 1 \Leftrightarrow (\exists j \mid j = next_{C'}(i) \text{ and } C' \in C_k)$; otherwise $T_k[i] = 0$. Since a classes at level k contain all the starting positions of a substrings of length k grouped according to substring equality, $T_k[i] = 1$ indicates the existence of a closed string at i .

At level k , T_k is initialized to all zeros. Let $C' = \{j_1, j_2, j_3, \dots, j_{|C'|}\}$ be an equivalence class such that $C' \in C_k$. Suppose, during refinement C' is partitioned. Let j_m , where $m \in [1..|C'|]$, be the first index removed from C' . If $prev_{C'}(j_m)$ exists, let $j_{m-1} = prev_{C'}(j_m)$ and set $T_k[j_{m-1}] \leftarrow k$, since $w[j_{m-1}..j_m + k - 1]$ forms a closed substring. Similarly, if $next_{C'}(j_m)$ exists, let $j_{m+1} = next_{C'}(j_m)$ and set $T_k[j_m] \leftarrow k$, since $w[j_m..j_{m+1} + k - 1]$ forms a closed substring. In both cases, the border of the closed substring is $b = w[j_{m-1}..j_{m-1} + k - 1]$ of length k . Recall that, by Fact 2 in Section 2.1, each class is represented as a doubly linked list with auxiliary pointers. Therefore, in Algorithm 3, we use the $prev_{C'}()$ and $next_{C'}()$ pointers to simulate the behavior of an ordered set.

To check if these closed substrings are maximal right-closed, we perform a constant time check by checking if $j_m + k = n$ or $(j_m + k < n \text{ and } w[j_{m-1} + k] \neq w[j_m + k])$,

and $j_{m+1} + k = n$ or ($j_{m+1} + k < n$ and $w[j_m + k] \neq w[j_{m+1} + k]$), respectively. The lines 11 and 16 in Algorithm 3 perform these comparisons. If they are maximal right-closed, then we add them to the $\mathcal{MRC}$ array.

Later in the refinement process, if j_{m-1} is removed from C' , the longer string $w[j_{m-1}..j_{m+1} + k - 1]$ with border b , also contains b at j_m . Hence, it is not a closed substring, even though $j_{m+1} = \text{next}_{C'}(j_{m-1})$. Moreover, at any index position, we cannot have a closed string with two different borders of the same length. Therefore, the string $w[j_{m-1}..j_{m+1} + k - 1]$ is either open or is closed with a longer border and would be found at a level greater than k in the algorithm.

The auxiliary array T_k is used conceptually to determine whether a candidate substring is closed at level k . Maintaining a separate boolean array for each level would require $\mathcal{O}(n^2)$ space. To achieve $\Theta(n)$ space complexity, we instead maintain a single integer array $T[1..n]$, initialized to all zeros. The array T acts as a level-dependent timestamp structure: the condition $T_k[i] = 1$ is represented by $T[i] = k$. Thus, if a closed substring with border length k is identified at position i during level k , we set $T[i] \leftarrow k$, as shown in lines 13 and 18 of Algorithm 3, regardless of whether the corresponding substring is maximal right-closed. Consequently, whether a candidate substring starting at position i has already been identified as closed at level k can be determined in constant time by testing whether $T[i] = k$. A clear analysis of the $\Theta(n)$ space complexity of the CMR Algorithm is presented in Theorem 12.1.1 [20]. Thus, we obtain the following result.

Theorem 3 *Algorithm 3 correctly computes the $\mathcal{MRC}$ array of a string $w[1..n]$ without pre-processing in $\mathcal{O}(n \log n)$ time and $\Theta(n)$ auxiliary space.*

4.3 Algorithm to Compute $\mathcal{C}(w)$ and MCSs

Algorithm 4 computes the compact representation $\mathcal{C}(w)$, as in (4), of all closed substrings of a string $w[1..n]$, by simply traversing all n lists in the $\mathcal{MRC}$ array (see example in Table 2). Since the $\mathcal{MRC}$ array is of size $\mathcal{O}(n \log n)$ we get Theorem 4.

Algorithm 4 Compute the compact representation $\mathcal{C}(w)$ using $\mathcal{MRC}$ array.

```

1: procedure COMPUTE- COMPACT- REPRESENTATION( $\mathcal{MRC}$ )
2:    $\mathcal{C} \leftarrow \emptyset$ 
3:   for  $i \leftarrow 1$  to  $n$  do
4:     for  $j \leftarrow 1$  to length of  $\mathcal{MRC}[i]$  do
5:        $(\ell r_j, \ell b_j) \leftarrow \mathcal{MRC}[i][j]$                                  $\triangleright \ell r_j = |r_j|, \ell b_j = |b_j|, \ell p_j = |p_j|$ 
6:       if  $j = 1$  then
7:          $\ell p_j \leftarrow 1$                                                $\triangleright (2)$ 
8:       else
9:          $(\ell r_{j-1}, \ell b_{j-1}) \leftarrow \mathcal{MRC}[i][j - 1]$ 
10:         $\ell p_j \leftarrow \ell r_j - \ell b_j + \ell b_{j-1} + 1$                    $\triangleright (2)$ 
11:        insert( $\mathcal{C}, (i, \ell p_j, \ell r_j)$ )
12:   return  $\mathcal{C}$                                                           $\triangleright \mathcal{C}$  is the compact representation as stated in (3)

```

Theorem 4 *Algorithm 4 correctly computes all $\mathcal{O}(n^2)$ closed substrings of a string $w[1..n]$ in a compact representation in $\mathcal{O}(n \log n)$ time.*

Table 2 Compact representation $\mathcal{C}(w)$ of $w[1..11] = \text{mississippi}$ with corresponding closed factors, maximal right-closed factors, and MCSs.

$C(w)$	All Closed Factors	Maximal Right Closed Factors	MCS?
(1, 1, 1)	m	m	Yes
(2, 4, 7)	$issi, issis, ississ, ississi$	$ississi$	Yes
(2, 1, 1)	i	i	Yes
(3, 5, 6)	$ssiss, ssissi$	$ssissi$	No
(3, 1, 2)	s, ss	ss	Yes
(4, 5, 5)	$sissi$	$sissi$	No
(4, 3, 3)	sis	sis	Yes
(4, 1, 1)	s	s	No
(5, 4, 4)	$issi$	$issi$	No
(5, 1, 1)	i	i	Yes
(6, 1, 2)	s, ss	ss	Yes
(7, 1, 1)	s	s	No
(8, 4, 4)	$ippi$	$ippi$	Yes
(8, 1, 1)	i	i	Yes
(9, 1, 2)	p, pp	pp	Yes
(10, 1, 1)	p	p	No
(11, 1, 1)	i	i	Yes

We show that the $\mathcal{MRC}$ array can also be used to compute all MCSs of a string $w[1..n]$ by performing a simple left to right scan of the array and conducting constant time checks of left extendibility, as presented in Algorithm 5. In particular, for each index i in w (where $1 \leq i \leq n$), the algorithm examines each maximal right-closed substring represented in $\mathcal{MRC}[i]$ and verifies whether the substring is also maximal left-closed using a constant time check $i = 1$ or $(i > 1 \text{ and } w[i - 1] \neq w[i + |r| - |b| - 1])$. If the condition is satisfied, the substring is added to the MCS list (see example in Table 2).

Scanning the $\mathcal{MRC}$ array requires at most $\mathcal{O}(n \log n)$ time, and each check for left extendibility is performed in constant time. Therefore, Algorithm 5 computes all MCSs in $\mathcal{O}(n \log n)$ time, establishing Theorem 5.

Algorithm 5 Compute all MCSs of $w[1..n]$ using the pre-computed $\mathcal{MRC}$ array.

```

1: procedure ( $w, \mathcal{MRC}$ )
2:    $mcsList \leftarrow \emptyset$ 
3:   for  $i \leftarrow 1$  to  $n$  do
4:     for  $j \leftarrow 1$  to  $\text{length of } \mathcal{MRC}[i]$  do
5:        $(\ell r, \ell b) \leftarrow \mathcal{MRC}[i][j]$   $\triangleright \ell r = |r|, \ell b = |b|$ 
6:       if  $i = 1$  or  $(i > 1 \text{ and } w[i - 1] \neq w[i + \ell r - \ell b - 1])$  then
7:          $\text{insert}(mcsList, w[i..i + \ell r - 1])$ 
8:   return  $mcsList$   $\triangleright mcsList$  contains all MCSs of  $w[1..n]$ 

```

Theorem 5 Algorithm 5 correctly computes all MCSs of a string $w[1..n]$ in $\mathcal{O}(n \log n)$ time.

Note that Badkobeh et al. [12] also achieve the same theoretical time complexity for computing MCSs. Their approach relies on a bottom-up traversal of a binary suffix tree [21] (i.e., a suffix tree transformed into one with at most two children) for $w[1..n]$ to compute all MCSs. In contrast our method employed in the $\mathcal{MRC}$ array construction (Algorithm 2), exploits the rise and fall of the LCP array values, which are tracked using a stack that effectively simulates the behavior of a bottom-up tree traversal to compute all maximal right closed substrings. Furthermore, although both approaches employ AVL trees to store and merge sorted lists, they differ in how these trees are augmented. This distinction arises because our work and that of Badkobeh et al. build on different results of Brodal et al.—namely, quasiperiodicity [18] and gapped repeats [22], respectively. Consequently, the augmentation schemes differ, leading to structurally distinct AVL trees. In addition, we employ an adapted version of the augmented AVL tree of Brodal et al., as refined by Kociumaka et al. [17], incorporating the UNION and CHANGELIST operations. As a result, the underlying data structures differ significantly, which in turn leads to non-trivial differences in the complexity analysis.

Finally, the main contribution of Algorithm 2 is the efficient computation of the $\mathcal{MRC}$ array, a data structure that facilitates the identification of all maximal right-closed strings, rather than all MCSs, in order to obtain a compact representation of all closed substrings—a result not presented in [12]. Algorithm 5 is then a straightforward application of this data structure.

We were unable to experimentally compare the performance of Algorithm 5 with that of [12]. Nevertheless, we expect our implementation to perform better in practice, particularly for long strings, due to its conceptual simplicity and the fact that it avoids the use of the space-intensive suffix tree.

5 MCSs in a Fibonacci Word

In this section, we study the occurrences of MCSs in a Fibonacci word (f_n), and provide a formula to compute the exact number of MCSs in f_n . Note that throughout this section, n refers to the n -th Fibonacci number or word, rather than to the string length.

Let us recall that a **Fibonacci** word f_n is defined recursively by $f_0 = 0$, $f_1 = 1$, and $f_n = f_{n-1}f_{n-2}$ for $n \geq 2$. We denote $F_n = |f_n|$, where F_n satisfies the Fibonacci recurrence relation $F_n = F_{n-1} + F_{n-2}$, with $F_0 = 1$ and $F_1 = 1$. We refer to the logical separation between f_{n-1} and f_{n-2} , in the string f_n , as the **boundary**.

Bucci et al. [23] study the open and closed prefixes of Fibonacci words and show that the sequence of open and closed prefixes of a Fibonacci word follows the Fibonacci sequence. De Luca et al. [24] explore the sequence of open and closed prefixes of a Sturmian word. In [6], the authors define and find the closed Ziv–Lempel factorization and classify closed prefixes of infinite m -bonacci words. In this section, we extend previous work on closed substrings in Fibonacci words by deriving a formula to compute the number of MCSs they contain.

For any integer $n \geq 2$, let $p_n = f_n[1..F_n - 2]$ denote the prefix of f_n excluding its last two symbols. Then, $f_n = p_n \delta_n$, where $\delta_n = 10$ if n is even and $\delta_n = 01$,

otherwise. Let $P_n = F_n - 2$ denote the length of p_n . The golden ratio is defined as $\phi = \frac{1+\sqrt{5}}{2}$, and it is known that the ratio of consecutive Fibonacci numbers converges to ϕ , i.e., $\lim_{n \rightarrow \infty} \frac{F_n}{F_{n-1}} = \phi$.

The non-singleton MCSs are either runs or maximal gapped repeats in which the arm occurs exactly twice [11]. For a Fibonacci word f_n , let $SM(f_n)$, $R(f_n)$, and $GM(f_n)$ denote the number of singleton MCSs, runs, and gapped-MCSs, respectively. Therefore, the total number of MCSs in f_n is given by $M(f_n) = SM(f_n) + R(f_n) + GM(f_n)$.

Lemma 6 *The no. of singleton MCSs in a Fibonacci word f_n , for $n \geq 4$, is:*

$$SM(f_n) = \begin{cases} F_{n-2} + F_{n-4} + 2, & \text{if } n \text{ is odd,} \\ F_{n-2} + F_{n-4}, & \text{if } n \text{ is even.} \end{cases} \quad (6)$$

Proof We prove the Lemma by induction on n . The base cases, $SM(f_4) = 3$ and $SM(f_5) = 6$, match the direct counts.

For the inductive step, assume that (6) holds for $k \geq 6$. By definition, $f_{k+1} = f_k f_{k-1}$. Every Fibonacci word f_k begins with a 10; if k is odd, it ends with 01 and if k is even, it ends with 10. By definition, the first and last character in f_k are singleton MCSs. Therefore, we examine whether the singleton MCSs at the boundary are retained in f_{k+1} .

Suppose $k + 1$ is even. By inductive hypothesis, $SM(f_k) = F_{k-2} + F_{k-4} + 2$ and $SM(f_{k-1}) = F_{k-3} + F_{k-5}$. Since k is odd, f_k ends with 1 and f_{k-1} begins with 1, their concatenation results in the string 11 at the boundary, which forms a new non-singleton MCS, and reduces the count of the singleton MCSs in f_{k+1} by two. Thus, we get $SM(f_{k+1}) = SM(f_k) + SM(f_{k-1}) - 2 = F_{k-1} + F_{k-3}$.

Suppose $k + 1$ is odd. By inductive hypothesis, $SM(f_k) = F_{k-2} + F_{k-4}$ and $SM(f_{k-1}) = F_{k-3} + F_{k-5} + 2$. Since k is even, f_k ends with 0 and f_{k-1} begins with 1, their concatenation results in the string 01 at the boundary, which does not reduce the count of singleton MCSs in f_{k+1} . Thus, we get $SM(f_{k+1}) = SM(f_k) + SM(f_{k-1}) = F_{k-1} + F_{k-3} + 2$. $\square$

Lemma 7 (Theorem 2 in [25]) *For $n \geq 4$, the number of runs in the Fibonacci word f_n is given by $R(f_n) = 2F_{n-2} - 3$.*

To compute $GM(f_n)$, we count the number of maximal gapped repeats whose arms occur exactly twice, as these are, by definition gapped-MCSs. In Theorem 6 we first show that the only gapped-MCSs in a Fibonacci word are occurrences of the substring 101 that are also maximal gapped repeats. Then, we count all such occurrences in f_n .

Lemma 8 (Theorem 1 in [26]) *Suppose uvu is a maximal gapped repeat in f_n that is also a suffix of f_n . Then, the arm u is a Fibonacci word.*

Lemma 9 *For $n \geq 5$, suppose uvu is a maximal gapped repeat in f_n that is also a suffix of f_n . Then uvu is not a gapped-MCS.*

Proof By Lemma 8, the arm of the maximal gapped repeat is a Fibonacci word $u = f_k$. Clearly, $k \neq n$.

If n is even, then the proper suffixes of f_n that are Fibonacci words are in the set $\mathcal{S} = \{f_k \mid k \bmod 2 = 0 \wedge 0 \leq k < n\}$. For any proper suffix $f_k \in \mathcal{S} \setminus \{f_0, f_2\}$, f_{k+2} is a suffix of f_n . By definition, $f_{k+2} = f_{k+1}f_k = f_k f_{k-1}f_k = f_k f_k f_{k-3}f_{k-2}$, the suffix $f_{k-1}f_k = f_k f_{k-3}f_{k-2}$ has length $F_k + F_{k-1} < 2F_k$ implying that the last two occurrences of f_k overlap. Therefore, any uvu with f_k as its arm, will have at least three occurrences of f_k making it an open string. For suffix $f_k \in \{f_0, f_2\}$ of f_n , for $n \geq 6$, $f_6 = 1011010110110$ is a suffix of f_n , and the last two occurrences of f_k are left-extendible, and so, it is not maximal. Moreover, any longer uvu with the arm f_k will have at least three occurrences of f_k making it an open string.

If n is odd, then $\mathcal{S} = \{f_k \mid k \bmod 2 = 1 \wedge 1 \leq k < n\}$. The argument is analogous for $f_k \in \mathcal{S} \setminus \{f_1\}$ and $f_k \in \{f_1\}$, respectively. $\square$

Lemma 10 (Theorem 2 in [26]) *For $n \geq 3$, suppose uvu is a maximal gapped repeat in f_n that is not a suffix of f_n . Then the arm u belongs to the set $\mathcal{A}_n = \{p_3, p_4, \dots, p_{n-2}\}$.*

Lemma 11 (Lemma 8 in [26]) *For $n \geq 4$, all the borders of p_n form the set $\mathcal{B}_n = \{p_3, p_4, \dots, p_{n-1}\}$.*

Lemma 12 *For $n \geq 5$, the border p_{n-1} of p_n is the longest proper cover of p_n .*

Proof By Lemma 11, we know that p_{n-1} is the longest border of p_n . $F_n = F_{n-1} + F_{n-2}$. For $n \geq 5$, $F_n - 4 = F_{n-1} + F_{n-2} - 4$, and so $P_n - 2 = P_{n-1} + P_{n-2}$. Hence, $P_n \leq 2 \cdot P_{n-1}$. Clearly, p_{n-1} is either an adjacent or an overlapping border of p_n . Therefore, p_{n-1} is the longest proper cover of p_n .

Lemma 13 (Lemma 2 in [27]) *Let u be a proper cover of x and let $v \neq u$ be a substring of x such that $|v| \leq |u|$. Then, v is a cover of $x \Leftrightarrow v$ is a cover of u .*

Lemma 14 *For $n \geq 5$, all covers of p_n form the set $\mathcal{C}_n = \{p_4, p_5, \dots, p_n\}$.*

Proof By Lemma 12, for $n \geq 5$, p_{n-1} is the longest proper cover of p_n . Similarly, p_{n-2} is the longest proper cover of p_{n-1} , p_{n-3} is the longest proper cover of p_{n-2} , and so on. Therefore, by Lemma 13, it follows that all covers of p_n belong to the set $\mathcal{C}_n = \{p_4, p_5, \dots, p_n\}$ for all $n \geq 5$. $\square$

Theorem 6 *For $n \geq 5$, every gapped-MCS in a Fibonacci word f_n corresponds to an occurrence of the substring 101 that is also a maximal gapped repeat.*

Proof All gapped-MCSs are maximal gapped repeats. By Lemma 9, for $n \geq 5$, all maximal gapped repeats that are also suffixes of f_n are not gapped-MCSs.

By Lemma 10, for $n \geq 3$, if uvu is a maximal gapped repeat in f_n that is not a suffix of f_n , then the arm u belongs to the set $\mathcal{A}_n = \{p_3, p_4, \dots, p_{n-2}\}$. By Lemma 14, all covers of p_n belong to the set $\mathcal{C}_n = \{p_4, p_5, \dots, p_n\}$.

Suppose there exists a maximal gapped $w = uvu$, where $u \in \mathcal{A}_n \setminus p_3$, that is a gapped-MCS. Assume n is odd. Then, $f_n = p_n \delta_n$, where $\delta_n = 01$, and $f_n[F_n - 1] = 0$. Since $u \in \mathcal{A}_n \setminus p_3$, it ends with 1. Clearly, u does not occur ending at position $F_n - 1$.

Now suppose n is even. By a similar reasoning, u does not occur ending at $F_n - 1$ since it ends with 01 and $f_n[F_n - 2] = f_n[F_n - 1] = 1$ (as $f_4 = 10110$ is a suffix of f_n). Hence, u ends within p_n . Since u is a cover of p_n , $w = uvu$ will contain at least three occurrences of u making it an open string. Therefore, there is no maximal gapped repeat of the form $w = uvu$, where $u \in \mathcal{A}_n \setminus p_3$, that is a gapped MCS.

Now we consider the final case, where $u = p_3 = 1$. In this case, we get 101 as the only gapped-MCS. Any longer maximal gapped repeat will have at least three occurrences of $u = p_3 = 1$, making it an open string — a contradiction. $\square$

Lemma 15 *The no. of gapped-MCSs in a Fibonacci word f_n , for $n \geq 5$, is:*

$$GM(f_n) = \begin{cases} F_{n-5}, & \text{if } n \text{ is odd,} \\ F_{n-5} + 1, & \text{if } n \text{ is even.} \end{cases} \quad (7)$$

Proof We prove the Lemma by induction on n . By direct counting for the base cases, we verify that $GM(f_5) = 1$ and $GM(f_6) = 2$. For the inductive step, assume that (7) holds for $k \geq 7$. For $n \geq 3$, $f_3 = 101$ is a prefix of every f_n , and by Theorem 6 every gapped-MCS in a Fibonacci word f_n , for $n \geq 5$, corresponds to an occurrence of the substring 101 that is also a maximal gapped repeat. In $f_{k+1} = f_k f_{k-1}$, any new occurrences of 101 in f_{k+1} can only occur beginning in f_k and ending in f_{k-1} . Moreover, an occurrence of 101 as a suffix of f_k or a prefix of f_{k-1} may be lost by extension. Below we examine all such cases.

Suppose $k + 1$ is odd. By inductive hypothesis, $GM(f_k) = F_{k-5} + 1$ and $GM(f_{k-1}) = F_{k-6}$. Moreover, f_k ends with the suffix $f_2 = 10$ and f_{k-1} begins with $f_3 = 101$, resulting in the formation of the string 101 crossing the boundary. Clearly, this string is not a gapped-MCS as it is right-extendible. The gapped-MCS 101 which is the prefix of f_{k-1} is no longer a gapped-MCS since f_k ends with a 0 making it left-extendible. Thus, we get $GM(f_{k+1}) = GM(f_k) + GM(f_{k-1}) - 1 = (F_{k-5} + 1) + F_{k-6} - 1 = F_{k-4}$.

Suppose $k + 1$ is even. By inductive hypothesis, $GM(f_k) = F_{k-5}$ and $GM(f_{k-1}) = F_{k-6} + 1$. f_k ends with a 1, so the prefix 101 of f_{k-1} is retained as a gapped-MCS. By Lemma 9 the suffix $f_3 = 101$ of f_k is not a gapped-MCS. Thus, we get $GM(f_{k+1}) = GM(f_k) + GM(f_{k-1}) = F_{k-4} + 1$. $\square$

Theorem 7 *The number of MCSs in a Fibonacci word f_n , denoted by $M(f_n)$, for $n \geq 5$, is given below in (8). Furthermore, $M(f_n) \approx 1.382F_n$.*

$$M(f_n) = \begin{cases} F_n + F_{n-2} - 1, & \text{if } n \text{ is odd,} \\ F_n + F_{n-2} - 2, & \text{if } n \text{ is even.} \end{cases} \quad (8)$$

Proof Adding Equations (6), (7) and the Equation in Lemma 7 and simplifying, we get (8). Next, we have $M(f_n) < F_n + F_{n-2} = \left(1 + \frac{F_{n-2}}{F_n}\right) F_n \approx \left(1 + \frac{1}{\phi^2}\right) F_n \approx 1.382F_n$ (since $\lim_{n \rightarrow \infty} \frac{F_{n-2}}{F_n} = \frac{1}{\phi^2}$).

6 Experimental Evaluation

In this section, we evaluate the performance of the two proposed algorithms—Algorithm 2 (SA & LCP based) and Algorithm 3 (CMR based)—on different types of strings: periodic, aperiodic, DNA and random. We also exhaustively generate all strings over small alphabets of size $\sigma = 2, 3$, and 4 and of length n for $1 \leq n \leq 20$, and compute the maximum number of maximal closed substrings (MCSs) they contain.

6.1 Implementation Details

The code was implemented in C++ and compiled using GCC 11.5.0 with `-O3` flag and is available at <https://github.com/neerjamhaskar/closedStrings>.

The implementations for computing the suffix array (SA) and the longest common prefix array (LCP) were sourced from the *libsais* library [28], which builds on the contributions of [29–32].

The AVL trees used in Algorithm 2 are implemented using the C++ `std::set` container. To maintain the logarithmic balancing properties of the AVL trees, the `std::set` container is implemented as a red-black tree, providing equivalent asymptotic guarantees and efficient memory management as the AVL trees. The `std::set` interface supports key operations such as `insert`, `set::upper_bound`, and `set::lower_bound`, all of which operate in $\mathcal{O}(\log n)$ time. These operations are particularly useful for efficiently computing the CHANGELIST, which records updates to successor and predecessor relationships between sets during UNION operations. Specifically, the `set::insert` operation merges elements into a set, while `set::upper_bound` and `set::lower_bound` enable fast determination and update of adjacency relationships within the merged sets. Note that, our independently developed CHANGELIST() implementation, is closely related to the implementation of partial covers available at <https://github.com/heurezjusz/Quasiperiods>, developed for [33].

A simple stack is employed to store pointers to the suffix sets being processed at various stages of Algorithm 2, allowing efficient access to both current and previously processed sets with minimal memory overhead. The implementation uses the C++ `std::stack` template to ensure type safety and flexibility during execution.

For Algorithm 3, the implementation of the CMR Algorithm for computing maximal repetitions was sourced from [34]. We modified this implementation to compute the *MRC* array. In both of our algorithms, the *MRC* array is implemented as `std::vector` of `std::list`, enabling constant-time access and insertion to either ends of the list.

6.2 Experiments

All experiments were carried out on a server with an Intel Xeon Gold 6426Y CPU (64 cores, 128 threads, 75 MiB L3 cache), 250 GiB RAM, running RHEL 9.5 (kernel version 5.14.0-503.26.1). In our experiments the running time of Algorithm 2 includes the pre-processing required to construct the suffix array (SA) and longest common prefix array (LCP) using the *libsais* library.

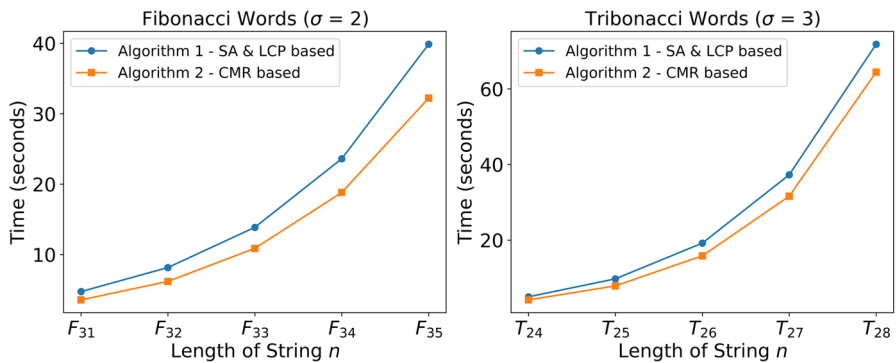


Fig. 3 Execution time comparison between Algorithm 2 (SA & LCP based) and Algorithm 3 (CMR based) on Fibonacci and Tribonacci words

We first run Algorithms 2 and 3 on well-studied Fibonacci and Tribonacci words which are known to be highly periodic. The Tribonacci word t_n is defined recursively as $t_n = t_{n-1}t_{n-2}t_{n-3}$ with $t_0 = 1$, $t_1 = 12$, and $t_2 = 1213$; the length of each Tribonacci word is denoted as $T_n = |t_n|$. As shown in Fig. 3, Algorithm 3 consistently outperforms Algorithm 2.

We then evaluate Algorithms 2 and 3 on Thue–Morse words and the digits of π (which have lesser number of periodic substrings and are non-repetitive). The Thue–Morse words tm_n are generated for our experiments using a Python implementation of the standard morphism $\mu(0) = 01$ and $\mu(1) = 10$ with $tm_0 = 0$; the length of each word is $TM_n = |tm_n| = 2^n$. The digits of π are regarded as an effectively aperiodic sequence for computational purposes. As shown in Fig. 4, Algorithm 2 outperforms Algorithm 3 on these string classes.

For our experiments on biological sequences, we evaluate both Algorithms 2 and 3 on four DNA datasets: prefixes of Human Chromosome 1, Human Chromosome 19, *Escherichia coli*, and Influenza. Each dataset consists of increasingly larger prefixes, from 5M up to 50M, in increments of 5M characters. Across all four datasets, Algorithm 2 consistently outperforms Algorithm 3, with the performance gap widening as the string length increases as shown in Fig. 5.

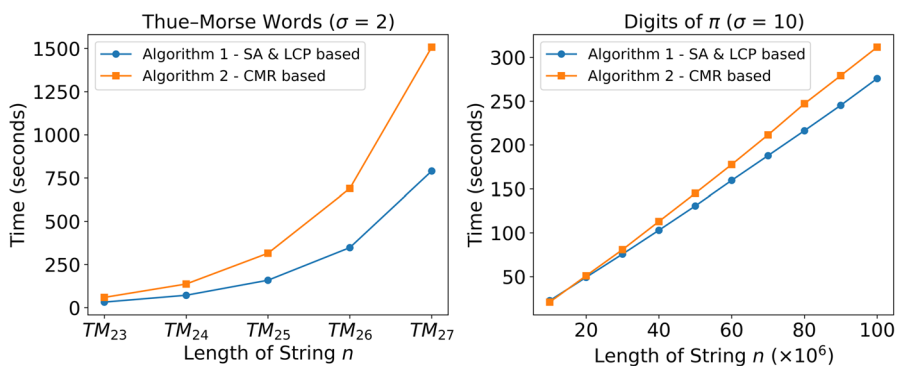


Fig. 4 Execution time comparison between Algorithm 2 (SA & LCP based) and Algorithm 3 (CMR based) on aperiodic Thue–Morse words and digits of π

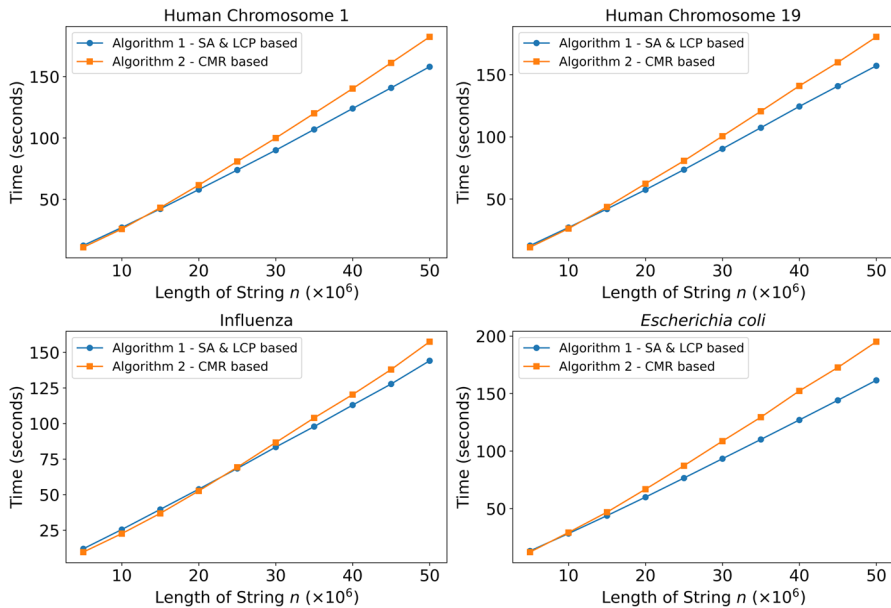


Fig. 5 Execution time comparison between Algorithm 2 (SA & LCP based) and Algorithm 3 (CMR based) on DNA sequences—prefixes of Human Chromosome 1, Human Chromosome 19, *Escherichia coli*, and Influenza

Next, we generate random strings over alphabets of size 2, 4, and 20 in increments of 10M, from 10M up to 100M, using the *Pizza&Chili* (<https://pizzachili.dcc.uchile.cl/experiments.html>) random string generator. For each string, we run both Algorithms 2 and 3 three times and report the median running time. Across all string lengths and alphabet sizes, we observe that Algorithm 2 consistently outperforms Algorithm 3 (Fig. 6).

Overall, although Algorithm 3 is slower than Algorithm 2 on most string classes considered, it offers greater potential for parallelization. In Algorithm 2, parallelism is largely limited to the initial partition of suffixes into at most σ sets. In contrast, as Algorithm 3 progresses through successive levels, the refinement process typically produces an increasing number of equivalence classes, thereby enabling a higher degree of parallel execution.

For $\sigma = 2, 3$ and 4, we generate all strings of length n for $1 \leq n \leq 20$ and compute their MCSs. The results are plotted in Fig. 7. As observed, the maximum number of MCSs increases with the size of the alphabet σ for strings of the same length. In total, the number of strings analyzed is equal to $\sum_{i=1}^{20} (2^i + 3^i + 4^i) \approx 1.47$ trillion. While the relatively small range of lengths considered makes it difficult to determine the precise mechanism by which σ affects the number of MCSs, the increasing separation between the curves in Fig. 7 suggests that the alphabet size is indeed a contributing factor to the maximum number of MCSs in a string of length n , in part because the number of singleton MCSs increase with σ .

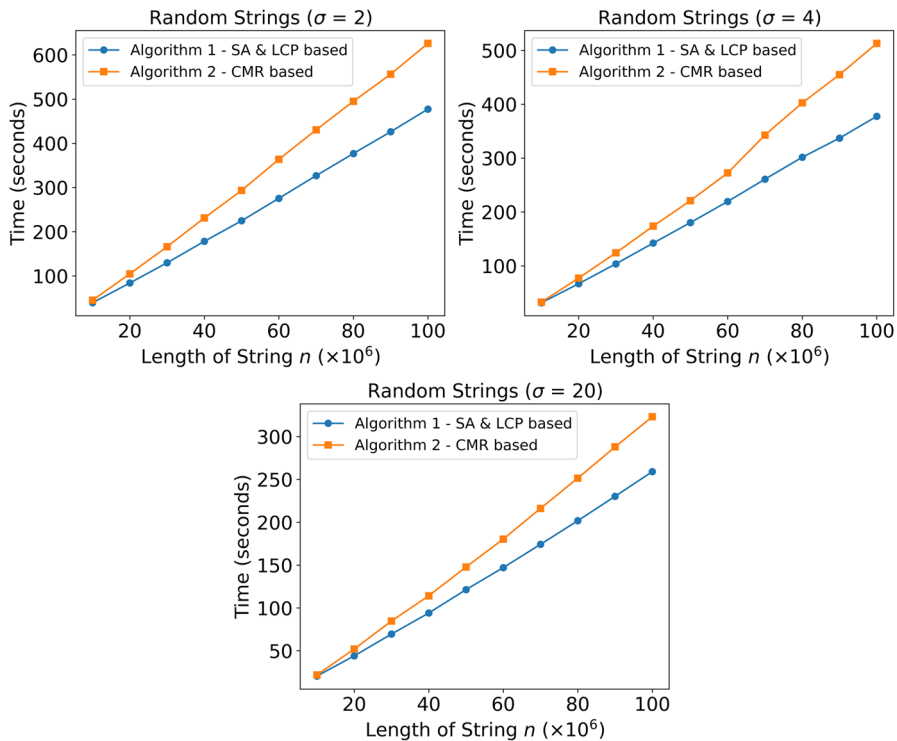


Fig. 6 Execution time comparison between Algorithm 1 (SA & LCP based) and Algorithm 2 (CMR based) on random strings

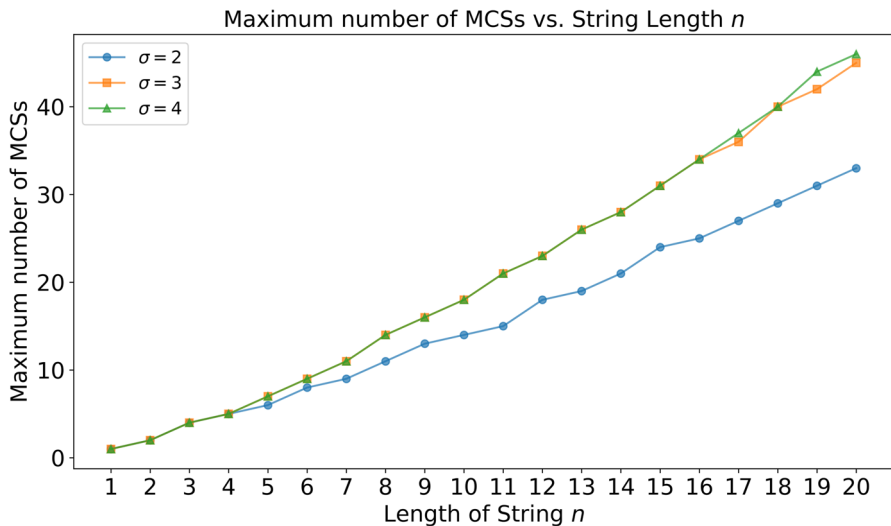


Fig. 7 Maximum number of MCSs in strings of length n , where $1 \leq n \leq 20$, for alphabets of size $\sigma = 2, 3$ and 4

7 Conclusion and Open Problems

We present a compact representation of all $\mathcal{O}(n^2)$ closed substrings of a string $w[1..n]$, requiring only $\mathcal{O}(n \log n)$ space. To compute this representation and identify all maximal closed substrings (MCSs), we introduce the *MRC* array and propose two $\mathcal{O}(n \log n)$ time algorithms (Algorithms 2 and 3) for its computation. Algorithm 2 requires pre-processing of the input string to compute the SA and LCP arrays, whereas Algorithm 3 does not require pre-processing and is based on the equivalence-class approach of the CMR algorithm. We also present an exact formula for computing the number of MCSs in Fibonacci words. Finally, we provide implementations and experimental analysis of the performance of Algorithm 2 and Algorithm 3. Our results show that Algorithm 2 performs best for most string classes, while Algorithm 3 outperforms Algorithm 2 for highly periodic strings.

Several questions remain open for future research. For example, Algorithm 2 allows parallelization only up to σ , whereas Algorithm 3 can be parallelized more aggressively; determining whether this advantage leads to better practical performance is an interesting challenge. Another intriguing direction is the algorithmic generation of strings that attain the maximum number of MCSs for arbitrary n and alphabet size σ . Moreover, extending the characterization of MCSs to m -bonacci words or more general Sturmian sequences presents a significant theoretical challenge. A deeper understanding of their structural and enumerative properties could lead to broader insights into the combinatorics of words and the design of efficient string algorithms.

Acknowledgements We thank Simon J. Puglisi for introducing us to the MCS problem using SA and LCP, which led to Theorem 5. We are grateful to the reviewers for their valuable feedback.

Author Contributions KJ, S : Initial conceptualization, Writing - original draft preparation, figures (including graphs), experiments and code implementation and string generation M.N : Funding acquisition & Supervision KJ, S & M.N : Methodology, Formal analysis and investigation, Writing - review and editing

Funding Neerja Mhaskar was funded by Natural Sciences & Engineering Research Council of Canada [Grant Number RGPIN-2024-06915].

Data Availability

- Bill Smyth's String Repository - <https://www.cas.mcmaster.ca/~bill/strings>
- *Pizza&Chili* Text Collection - <https://pizzachili.dcc.uchile.cl/texts.html>
- Digits of π - <https://calculat.io/storage/pi/100m.zip>
- Human Chromosome 1 - https://www.ncbi.nlm.nih.gov/nuccore/NC_000001.11

Code availability

<https://github.com/neerjamhaskar/closedStrings>

References

1. Fici, G.: A classification of Trapezoidal words. In: Ambrož, P., Holub, Masáková, Z. (eds.) Proceedings of the 8th International Conference WORDS 2011, Prague, Czech Republic, 12–16 September 2011. Electronic Proceedings in Theoretical Computer Science, vol. 63, pp. 129–137. Open Publishing Association, Online (2011). <https://doi.org/10.4204/EPTCS.63.18>
2. Badkobeh, G., Bannai, H., Goto, K., I, T., Iliopoulos, C.S., Inenaga, S., Puglisi, S.J., Sugimoto, S.: Closed factorization. In: Holub, J., Zdařek, J. (eds.) Proceedings of PSC 2014, pp. 162–168. Czech

- Technical University in Prague, Czech Republic (2014). <https://www.stringology.org/papers/PSC2014.pdf>
3. Bannai, H., Inenaga, S., Kociumaka, T., Lefebvre, A., Radoszewski, J., Rytter, W., Sugimoto, S., Waleń, T.: Efficient algorithms for longest closed factor array. In: String Processing and Information Retrieval, pp. 95–102. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-23826-5_10
 4. Badkobeh, G., Bannai, H., Goto, K., I, T., Iliopoulos, C.S., Inenaga, S., Puglisi, S.J., Sugimoto, S.: Closed factorization. *Discr. Appl. Math.* **212**, 23–29 (2016). <https://doi.org/10.1016/j.dam.2016.04.009>
 5. Alamro, H., Alzamel, M., Iliopoulos, C.S., Pissis, S.P., Watts, S., Sung, W.-K.: Efficient identification of k-closed strings. In: Boracchi, G., Iliadis, L., Jayne, C., Likas, A. (eds.) Engineering Applications of Neural Networks, pp. 583–595. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-65172-9_49
 6. Jahannia, M., Mohammad-Noori, M., Rampersad, N., Stipulanti, M.: Closed Ziv-Lempel factorization of the m-bonacci words. *Theoret. Comput. Sci.* **918**, 32–47 (2022). <https://doi.org/10.1016/j.tcs.2022.03.019>
 7. Parshina, O., Zamboni, L.Q.: Open and closed factors in Arnoux-Rauzy words. *Adv. Appl. Math.* **107**, 22–31 (2019). <https://doi.org/10.1016/j.aam.2019.02.007>
 8. Badkobeh, G., Fici, G., Lipták, Z.: On the number of closed factors in a word. In: Dediu, A.-H., Formenti, E., Martín-Vide, C., Truthe, B. (eds.) Language and Automata Theory and Applications, pp. 381–390. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-15579-1_29
 9. Parshina, O., Puzynina, S.: Finite and infinite closed-rich words. *Theoret. Comput. Sci.* **984**, 114315 (2024). <https://doi.org/10.1016/j.tcs.2023.114315>
 10. Mieno, T., Takahashi, S., Seto, K., Horiyama, T.: Online and offline algorithms for counting distinct closed factors via sliding suffix trees. In: Kráľovič, R., Kůrková, V. (eds.) SOFSEM 2025: Theory and Practice of Computer Science, pp. 172–183. Springer, Cham (2025). https://doi.org/10.1007/978-3-031-82697-9_13
 11. Badkobeh, G., De Luca, A., Fici, G., Puglisi, S.J.: Maximal closed substrings. In: Arroyuelo, D., Poblete, B. (eds.) String Processing and Information Retrieval, pp. 16–23. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-20643-6_2
 12. Badkobeh, G., De Luca, A., Fici, G., Puglisi, S.J.: Finding maximal closed substrings. *Theoret. Comput. Sci.* **1060**, 115628 (2026). <https://doi.org/10.1016/j.tcs.2025.115628>
 13. Kosolobov, D.: Closed Repeats (2024). <https://doi.org/10.48550/arXiv.2410.00209>
 14. Jain, S.K., Mhaskar, N., Badkobeh, G., Radoszewski, J., Tonellotto, N.: Efficient computation of closed substrings. In: Baeza-Yates, R. (ed.) String Processing and Information Retrieval, pp. 172–187. Springer, Cham (2026). https://doi.org/10.1007/978-3-032-05228-5_15
 15. Mhaskar, N., Smyth, W.F.: String covering: A survey. *Fund. Inform.* **190**(1), 17–45 (2023). <https://doi.org/10.3233/FI-222164>
 16. Crochemore, M.: An optimal algorithm for computing the repetitions in a word. *Inf. Process. Lett.* **12**(5), 244–250 (1981). [https://doi.org/10.1016/0020-0190\(81\)90024-7](https://doi.org/10.1016/0020-0190(81)90024-7)
 17. Kociumaka, T., Pissis, S.P., Radoszewski, J., Rytter, W., Waleń, T.: Fast algorithm for partial covers in words. *Algorithmica* **73**(1), 217–233 (2015). <https://doi.org/10.1007/s00453-014-9915-3>
 18. Brodal, G.S., Pedersen, C.N.S.: Finding maximal quasiperiodicities in strings. In: Giancarlo, R., Sankoff, D. (eds.) Combinatorial Pattern Matching, pp. 397–411. Springer, Berlin, Heidelberg (2000). https://doi.org/10.1007/3-540-45123-4_33
 19. Brown, M.R., Tarjan, R.E.: A fast merging algorithm. *J. ACM* **26**(2), 211–226 (1979). <https://doi.org/10.1145/322123.322127>
 20. Smyth, B.: Computing Patterns in Strings. ACM Press Bks. Pearson Addison-Wesley, Harlow, Essex, England (2003). <https://books.google.ca/books?id=iKR0EewiCu4C>
 21. Weiner, P.: Linear pattern matching algorithms. In: 14th Annual Symposium on Switching and Automata Theory (swat 1973), pp. 1–11. (1973). <https://doi.org/10.1109/SWAT.1973.13>
 22. Brodal, G.S., Lyngsø, R.B., Pedersen, C.N.S., Stoye, J.: Finding maximal pairs with bounded gap. In: Crochemore, M., Paterson, M. (eds.) Combinatorial Pattern Matching, pp. 134–149. Springer, Berlin, Heidelberg (1999)
 23. Bucci, M., De Luca, A., Fici, G.: Enumeration and structure of Trapezoidal words. *Theoret. Comput. Sci.* **468**, 12–22 (2013). <https://doi.org/10.1016/j.tcs.2012.11.007>
 24. De Luca, A., Fici, G., Zamboni, L.Q.: The sequence of open and closed prefixes of a Sturmian word. *Adv. Appl. Math.* **90**, 27–45 (2017). <https://doi.org/10.1016/j.aam.2017.04.007>

25. Kolpakov, R., Kucherov, G.: On maximal repetitions in words. In: Ciobanu, G., Păun, G. (eds.) *Fundamentals of Computation Theory*, pp. 374–385. Springer, Berlin, Heidelberg (1999). https://doi.org/10.1007/3-540-48321-7_31
26. Yamane, K., Nakashima, Y., Seto, K., Horiyama, T.: Maximal α -gapped repeats in a Fibonacci string. In: Kráľovič, R., Kůrková, V. (eds.) *SOFSEM 2025: Theory and Practice of Computer Science*, pp. 337–350. Springer, Cham (2025). https://doi.org/10.1007/978-3-031-82697-9_25
27. Moore, D., Smyth, W.F.: An optimal algorithm to compute all the covers of a string. *Inf. Process. Lett.* **50**(5), 239–246 (1994). [https://doi.org/10.1016/0020-0190\(94\)00045-X](https://doi.org/10.1016/0020-0190(94)00045-X)
28. Grebnov, I.: *libsais: A library for linear time suffix array, longest common prefix array and Burrows-Wheeler transform construction based on induced sorting algorithm*. GitHub repository. Version 2.10.2, last updated June 12, 2025 (2021–2025). <https://github.com/IlyaGrebnov/libsais>
29. Kärkkäinen, J., Manzini, G., Puglisi, S.J.: Permuted longest-common-prefix array. In: Kucherov, G., Ukkonen, E. (eds.) *Combinatorial Pattern Matching*, pp. 181–192. Springer, Berlin, Heidelberg (2009). https://doi.org/10.1007/978-3-642-02441-2_17
30. Nong, G., Zhang, S., Chan, W.H.: Two efficient algorithms for linear time suffix array construction. *IEEE Trans. Comput.* **60**(10), 1471–1484 (2011). <https://doi.org/10.1109/TC.2010.188>
31. Timoshevskaya, N., Feng, W.: Sais-opt: On the characterization and optimization of the sa-is algorithm for suffix array construction. In: *2014 IEEE 4th International Conference on Computational Advances in Bio and Medical Sciences (ICCABS)*, pp. 1–6 (2014). <https://doi.org/10.1109/ICCABS.2014.6863917>
32. Xie, J.Y., Nong, G., Lao, B., Xu, W.: Scalable suffix sorting on a multicore machine. *IEEE Trans. Comput.* **69**(9), 1364–1375 (2020). <https://doi.org/10.1109/TC.2020.2972546>
33. Czajka, P., Radoszewski, J.: Experimental evaluation of algorithms for computing quasiperiods. *Theoret. Comput. Sci.* **854**, 17–29 (2021). <https://doi.org/10.1016/j.tcs.2020.11.033>
34. Franek, F., Smyth, W.F., Xiao, X.: A note on Crochemore’s repetitions algorithm: A fast, space-efficient approach. In: *Proceedings of PSC 2002*, pp. 36–43. Department of Computer Science and Engineering, Czech Technical University in Prague, Czech Republic (2002). <https://www.stringology.org/event/2002/p5.html>

Publisher’s Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.